Agentic Time Machine as an Infrastructure for Future-Event Forecasting

Jingyi Chai[1]*, **Bingyang Zheng**[1]*, **Xiangrui Liu**[1], **Hao Lu**[1],
Zihang Zhou[1], **Tianchen Wang**[1], **Kemeng Zhang**[1], **Siheng Chen**[1] †
[1]Shanghai Jiao Tong University

Abstract

Forecasting future events is a critical challenge for large language model (LLM) agents, spanning domains from elections and monetary policy to financial markets. However, evaluating progress on this task presents a fundamental trade-off between efficiency and environment fidelity. While live evaluation benchmarks suffer from an inherently slow feedback loop, existing retrospective replays typically restrict agents to static, pre-frozen databases that sacrifice the environmental realism of actual deployments. To tackle this issue, we introduce Agentic Time Machine (TM), an infrastructure that approximates the pre-cutoff web state at any chosen past time by filtering post-cutoff content. Leveraging this evaluation infrastructure, we further propose a planner-solver-aggregator multi-agent framework that breaks each question into diverse analytical angles, gathers evidence in parallel, and combines the results into a single forecast. Experiments show that offline scores under TM correlate strongly with live FutureX scores, validating that TM offers a fast and reliable sandbox for forecasting-agent evaluation. On FutureX-Past and Polymarket evaluated under TM, our framework achieves the highest score among strong closed-book, tool-augmented, and self-consistency baselines. On the live FutureX leaderboard, our system achieves the best average rank over four consecutive weeks, ranking 1st in May Week 1.

1 Introduction

Forecasting real-world events such as election outcomes, monetary-policy decisions, sports results, and asset price movements remains a critical challenge for large language model (LLM) agents (Yang et al., 2024; Halawi et al., 2024). Unlike deep information retrieval tasks (Wei et al., 2025), the correct answer for a forecasting problem does not yet exist in any online source. The agent should collect useful evidence from many noisy sources, account for uncertainty, and reason about the potential outcomes. Recent live benchmarks such as FutureX (Zeng et al., 2025), Prophet Arena (Yang et al., 2025), and ForecastBench (Karger et al., 2025) facilitate evaluation of prediction capacity by releasing fresh questions periodically and scoring predictions once the real-world events resolve.

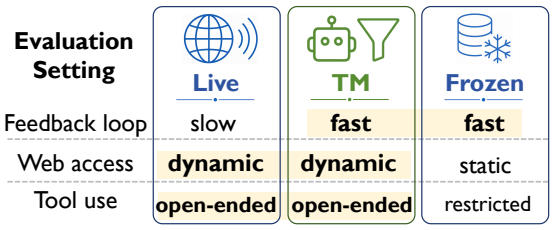


Figure 1: Comparison of evaluation settings for forecasting agents. Our Agentic Time Machine (TM) uniquely combines a fast feedback loop with dynamic, open-ended web interactions.

However, pushing the boundaries of real-world forecasting tasks presents a fundamental trade-off between evaluation efficiency and environment fidelity. As illustrated in Figure 1, there are two paradigms of existing evaluation benchmarks denoted as Live and Frozen. On one hand, **Live** evaluation environments (Zeng et al., 2025; Yang et al., 2025; Karger et al., 2025; Nechepurenko and Shuvalov, 2026) offer dynamic web access and open-ended tool use, yet they are constrained by an inherently slow feedback loop, where researchers must wait days or weeks for real-world event resolution. On the other hand, to accelerate development cycles, alternative benchmarks explore a **Frozen** replay paradigm (Chandak et al., 2026; Ye et al., 2024; Goel et al., 2026) or closed-book inference (Dai et al., 2025; Tan et al., 2025). Although these methods provide a fast feedback loop, they restrict the agent to retrieving information from a frozen database, fundamentally sacrificing the

*Equal contribution.
†Corresponding author: sihengc@sjtu.edu.cn

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 46016–46035
October 24-29, 2026 ©2026 Association for Computational Linguistics

environmental realism of actual deployments.

To address this challenge, we propose *Agentic Time Machine* (TM), an LLM-based evaluation infrastructure that achieves a fast feedback loop while preserving dynamic web access. By actively filtering post-cutoff or answer-revealing content from web tools, TM effectively simulates the open web as it stood at any chosen historical prediction point, preventing answer leakage (Lahib et al., 2026) during retrospective replay. Moreover, to improve forecasting ability, we build a planner-solver-aggregator multi-agent system that decomposes each question into a small set of analytical angles, gathers evidence along those different paths in parallel, and combines these findings into a single final forecast.

We validate both components through extensive empirical evaluation. We first compare agent performance with and without TM across five LLM backbones, where applying TM substantially lowers scores on prediction tasks and shows that our infrastructure effectively reduces answer leakage. Importantly, we find that offline scores achieved under the Time Machine correlate strongly with live leaderboard performance, establishing our infrastructure as a reliable proxy for real-world prediction testing. Leveraging TM, our forecasting system achieves the highest overall scores on two offline prediction datasets. On the public FutureX live leaderboard, the same system ranks among the leading submissions across four consecutive weeks. Together, these results suggest that fast and leakage-reduced local replay is a useful evaluation infrastructure, and planner-solver-aggregator multi-agent decomposition improves forecasting under both offline and live settings.

In summary, our contributions are as follows:

- **Agentic Time Machine**, an LLM-based evaluation infrastructure that simulates the web as it stood at a chosen point in time, enabling fast and reproducible local replay of forecasting tasks without exposing the resolved answer to the forecasting agent (Section 3).
- **A planner-solver-aggregator multi-agent forecasting system** in which a planner decomposes each question into diverse analytical angles, parallel solvers gather evidence, and an aggregator reconciles the resulting views, with no training and no extra tools beyond a single-agent baseline (Section 4).
- **Comprehensive empirical evidence** showing that Time Machine provides a fast and reliable infrastructure for local forecasting evaluation, and that our planner-solver-aggregator framework improves forecasting performance across offline and live settings (Section 5).

2 Related Work

2.1 LLM Prediction Benchmarks

Continuously updated forecasting benchmarks. A growing line of work evaluates LLMs on upcoming future events rather than retrospective question answering, establishing a **Live** evaluation paradigm. ForecastBench (Karger et al., 2025) is a dynamic benchmark for forecasting under genuine uncertainty, while FutureX (Zeng et al., 2025) further emphasizes the agentic setting by evaluating tool-using LLM agents on diverse live prediction tasks. Prophet Arena (Yang et al., 2025) studies predictive intelligence in a similarly live setup, and DailyOracle (Dai et al., 2025) continuously generates future-oriented evaluation instances from daily news to track temporal generalization. Although these live benchmarks offer authentic dynamic web access and open-ended tool use, they are constrained by an inherently slow feedback loop, as improving a system requires waiting days or weeks for real-world event resolution.

Retrospective replay and pastcasting benchmarks. To shorten this loop, another line of work implements a **Frozen** replay paradigm by replaying historical forecasting questions under temporally constrained conditions. Bench to the Future (Wildman et al., 2025) converts resolved questions into an offline benchmark with pre-collected evidence, and FutureSim (Goel et al., 2026) replays world events chronologically so that agents observe information as it unfolds over time. Similarly, MIRAI (Ye et al., 2024) evaluates event forecasting in a temporally grounded setting. While these retrospective benchmarks enable a fast feedback loop, they typically rely on frozen web snapshots or simulated replay environments that prevent agents from dynamically interacting with the vast real-time web information, thereby lacking dynamic web access and enforcing restricted tool use. Moreover, despite date-restricted search, naive retrospective evaluation remains highly vulnerable to post-hoc answer leakage through updated web snippets and pages with rolling content (Lahib et al., 2026). Unlike these approaches, our Agentic Time Machine (TM) retains the efficiency of local replay while preserv-

ing open-ended, dynamic web interaction and explicitly preventing answer leakage.

2.2 Multi-Agent Forecasting Systems

Recent work has explored multiple agents or multiple sampled reasoning paths. A simple form is self-consistency (Wang et al., 2023), which aggregates independently sampled solutions but mainly diversifies answers rather than the evidence-gathering process itself. More forecasting-specific systems go further. AIA Forecaster (Alur et al., 2025) combines agentic search with a supervisor agent that reconciles forecasts and applies calibration, while MAEPS (Jin et al., 2026) simulates human expert-team collaboration through specialized agents that gather evidence from different professional dimensions and reconcile temporally conflicting information. MiroFlow (Su et al., 2026) provides broader evidence that careful orchestration can strengthen complex research-style agent systems. While these prior works highlight the value of multi-agent coordination, they often rely on centralized search without explicitly diversifying retrieval perspectives. In contrast, our planner-solver-aggregator framework decomposes each question into a small set of analytical angles, drives parallel solvers along distinct evidence paths, and aggregates their reports into one final forecast.

3 Offline Prediction Evaluation with Agentic Time Machine

To enable fast iteration without waiting for real-world resolution, we build a controlled local test bed on top of the FutureX historical archive. The dataset itself is not a contribution of this paper; the contribution is the Time Machine filter that makes local replay faithful to the agentic setting that the live benchmark targets.

3.1 Problem Formulation

A future-event forecasting instance is a tuple $(q, t, a^\star)$, where q is a question about a real-world event, t is the question's *cutoff time*, and $a^\star$ is the ground-truth answer that becomes publicly known only after the event resolves at some time $t_{\text{res}} > t$. Given q and an information environment $\mathcal{E}$ (the body of web evidence the agent can reach via web tools), an LLM agent π outputs a prediction $\hat{a} \sim \pi(q, \mathcal{E})$, scored against $a^\star$ by a metric $r(\hat{a}, a^\star) \in [0, 1]$. Unlike standard QA, the resolved outcome is not yet publicly known at time t; the forecasting objective is therefore defined with respect to the *pre-cutoff* environment $\mathcal{E}_t$ that would have been visible at time t:

$$\max_{\pi} \; \mathbb{E}_{(q,t,a^\star)\sim\mathcal{D},\, \hat{a}\sim\pi(q,\mathcal{E}_t)} \big[\, r(\hat{a}, a^\star) \,\big]. \quad (1)$$

Since $\mathcal{E}_t$ is not directly accessible at evaluation time $t' > t$, local evaluation must substitute some approximation $\hat{\mathcal{E}} \approx \mathcal{E}_t$, and this choice shapes what is actually measured:

- **Live web** $\hat{\mathcal{E}} = \mathcal{E}_{t'>t}$: contains content created after t that may reveal $a^\star$; the task collapses from forecasting into retrieval.
- **Frozen snapshot** $\hat{\mathcal{E}} = \mathcal{E}_{\text{frozen}} \subset \mathcal{E}_t$: a small pre-curated subset of $\mathcal{E}_t$ avoids leakage but its coverage and interaction pattern no longer match the live web that agents are deployed against.
- **Time Machine** $\hat{\mathcal{E}} = \mathcal{E}_{\text{TM}} \approx \mathcal{E}_t$: apply an LLM-based leakage filter on the live web to remove content dated after t or that exposes $a^\star$, so that the agent's effective environment approximates $\mathcal{E}_t$ while preserving open-ended tool interaction.

3.2 Agentic Time Machine

Agentic Time Machine instantiates the environment $\mathcal{E}_{\text{TM}}$ on top of the live web. Ideally, we would retain only those items in $\mathcal{E}_{t'>t}$ that were already retrievable with the same content at time t:

$$\mathcal{E}^\star_{\text{TM}} = \{x \in \mathcal{E}_{t'>t} : x \text{ retrievable at } t\,\} \approx \mathcal{E}_t. \quad (2)$$

Directly determining whether an item was retrievable at t is impractical, so we approximate Equation 2 with a binary leakage filter $\phi(x \mid q, t, a^\star)$, conditioned on the question q, its cutoff t, and the resolved answer $a^\star$. Rather than deciding whether x was literally retrievable at t, the filter looks for two tractable signs of leakage. A *direct answer leak* fires when the item states or strongly implies $a^\star$. A *post-date info leak* fires when the item carries a time signal later than t, such as a publish date, a last-update date, or an in-page event date. An item is kept only if neither sign is present.

We assume the agent reaches the web only through a fixed set of web tools, in our implementation a web-search tool and a page-content tool, each with its own tool-specific prompt for the LLM filter (see Appendix A.2 for more details). Each



Figure 2: Agentic Time Machine filters web results to approximate the information environment available at a question's cutoff time. In this example, snippets that directly reveal the answer or contain post-cutoff information are removed, leaving only safe pre-cutoff evidence for forecasting.

tool call returns items $\mathcal{X} \subseteq \mathcal{E}_{t'>t}$, of which the agent actually sees

$$\mathcal{X}^{\mathrm{TM}} = \{\, x \in \mathcal{X} : \phi(x \mid q, t, a^{\star}) = \mathrm{KEEP} \,\}. \quad (3)$$

Aggregating Equation 3 over every tool call in a trajectory gives the agent's effective environment $\hat{\mathcal{E}} = \mathcal{E}_{\mathrm{TM}} = \bigcup_{\mathrm{calls}} \mathcal{X}^{\mathrm{TM}}$, an approximation but not in general an equality with the ideal $\mathcal{E}^{\star}_{\mathrm{TM}}$: a pre-cutoff article that strongly hints at the resolved answer can be over-blocked, and a post-cutoff page that happens to expose neither leakage sign can slip through. We characterize both error modes empirically in Appendix A.5.

Concretely, Agentic Time Machine wraps every web-tool call before the result reaches the forecasting agent. For search, the LLM filter checks each returned title and snippet. For page visits, it checks the retrieved page content. In both cases, the filter asks whether the content is outside the allowed time range, or whether it directly reveals the answer to the question. Items that fail either check are removed, and only the remaining items are passed to the agent. A typical case is sketched in Figure 2 on a Coventry-vs-Leicester forecasting question dated January 17, 2026. The filter drops a snippet titled "Coventry 2-1 Leicester" as a direct answer leak and drops a January 19 post-match interview as a post-date info leak.

Why an LLM filter instead of a rule-based date cutoff. A simpler design is to filter each retrieved URL by a single date stamp. We find this insufficient because a single date stamp cannot describe a page whose content evolves over time. Search engines attribute one heuristic date to each URL (some combination of creation, first-indexing, and in-page dates), which is not the same as the date on which the answer-bearing content was actually written. Many forecasting-relevant pages have a stable URL and an old indexed date but rolling body content. The Yahoo Finance historical-prices page for a stock such as AAPL is one such page: its URL and indexed date are stable, yet its in-snippet OHLC table extends to whichever date the reader queries, including the very date asked about by a forecasting question. So a date filter passes the URL through, but the snippet still reveals the high of day. A pre-cutoff news article later edited to add the resolved outcome behaves similarly.

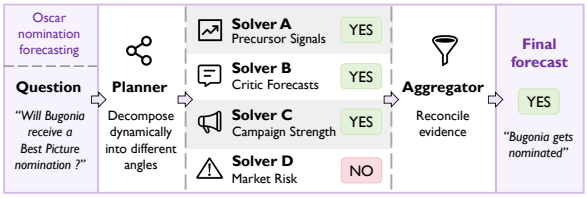


Figure 3: Overview of our planner-solver-aggregator multi-agent forecasting framework, illustrated on a Best Picture nomination question.

4 Planner-Solver-Aggregator Multi-Agent Framework

Forecasting an unresolved event is rarely a single-path reasoning problem. The same question can usually be approached from several legitimate angles, such as historical base rates and analogies, structural causal drivers, recent leading indicators, and contrarian risks. Each angle exposes a different slice of the relevant evidence. A capable forecaster typically does not commit to one angle in advance, but explores several in parallel and then reconciles what they imply.

Our framework follows this structure, as shown in Figure 3. A planner decides which angles to explore for the question at hand. Several solver agents cover those angles in parallel, each with its own search and reasoning. A final aggregator rec-

onciles their reports into a single grounded answer. All three roles are equipped with web tools.

Planner. Given the prediction question, the planner performs a few exploratory searches and produces a JSON plan with (a) a short task understanding, (b) the dominant causal factors, (c) historical patterns it has confirmed, and (d) 2–4 solver profiles. Each role profile defines a unique angle, a factor path the solver should follow, the search focus, and what the solver must deliver. The planner also writes aggregation advice.

Solvers. The 2–4 solvers run in parallel on separate threads. Each receives only the planner's understanding, shared key factors, and its own spec. The solver is instructed to use tools actively, ground its claims in evidence rather than priors, and produce a structured JSON report. Because each solver follows a distinct angle, the system can cover more relevant evidence without forcing one agent to pursue every possible line of reasoning.

Aggregator. The aggregator receives the planner output and all solver reports, identifies points of agreement and conflict, runs additional targeted searches when needed, and writes a free-form synthesis; the final line matches the question's required answer format.

Our framework adds no training, no fine-tuning, and no extra tools beyond what a single-agent baseline already has. Its main design choices are (i) a planner that first researches the prediction problem and assigns plausible forecasting angles, (ii) parallel angle-conditioned solvers that gather evidence from different paths, and (iii) an aggregator that reconciles individual views.

5 Experiments

5.1 Setup

Datasets. For offline evaluation, we use two historical question pools replayed via Agentic Time Machine. The first pool is a 61-question subset sampled from the FutureX historical archive (Zeng et al., 2025), which we refer to as FutureX-Past. It covers all four official difficulty levels: 17 at L1 (few options), 17 at L2 (multi-option with non-trivial domain context), 9 at L3 (compositional questions that chain several intermediate facts), and 18 at L4 (quantitative or open-ended). The second pool comprises 80 L2-style multi-option questions sampled from Polymarket, a real-money prediction market covering politics, finance, sports, and pop culture. We additionally evaluate our framework on the live, dynamic FutureX leaderboard, where real-world questions are still unresolved at submission time, thus Time Machine is not required for the live setting. Dataset construction details for both pools are in Appendix B, and a deeper analysis of our Time Machine is in Appendix A.

Baselines. We evaluate three categories of methods, all using a temperature of 1.0: (1) Models, six closed-book LLMs without tool access; (2) Agents, the same LLMs equipped with two Time Machine-wrapped tools (web search and page fetching) via a ReAct-style loop (Yao et al., 2023); and (3) Multi-Agent, including self-consistency (SC@N for $N \in \{3, 5\}$), which run the agent forecaster N times and majority-vote the final answer (Wang et al., 2023), as well as our planner-solver-aggregator framework, all of which utilize a GPT-5.4-medium (OpenAI, 2026) backbone and two web tools. For most configurations, we report the average score of three independent runs. However, for SC@N, we report a single ensemble run, as it already incorporates N forecasts. See web tool details in Appendix A.1.

Metric. We follow the official FutureX scoring protocol (Zeng et al., 2025). For L1 and L2, we report set-level F1, which reduces to exact match for single-option questions. For L3 and L4, an LLM judge evaluates predictions against resolved answers using three labels (Yes/Partial Correct/No); numeric L4 items additionally receive a graded distance score, $\max(0, 1 - ((\hat{y} - y))^2)$, to reward near-miss predictions. All per-question scores lie in $[0, 1]$. For FUTUREX-PAST, we report both per-level means (m_ℓ) and the weighted aggregate score $0.1m_1 + 0.2m_2 + 0.3m_3 + 0.4m_4$, which emphasizes harder levels. For Polymarket, we report mean set-level F1 across the 80 L2-style items. All scores are scaled by 100 for readability.

5.2 Agentic Time Machine Validation

Table 1 shows how the Agentic Time Machine affects performance across five different backbones. Without the Time Machine, most models achieve a weighted score between 60 and 69. This high performance occurs because agents can often find the final answers directly on the live web. However, enabling the Time Machine consistently reduces these scores by 25 to 37 points across different model families and sizes. The consistency of this drop suggests that the Time Machine is effectively blocking a common source of information leakage. Based on these results, we apply the Agentic

Model	w/o TM	w/ TM	Δ
Kimi-K2.5 (Kimi Team et al., 2026)	60.64	27.13	-33.51
GLM-5 (GLM-5-Team et al., 2026)	67.45	30.94	-36.51
DeepSeek-V4-Flash (DeepSeek AI, 2026)	69.11	35.02	-34.09
GPT-5.4-medium (OpenAI, 2026)	64.39	33.13	-31.26
Claude-4.6-Sonnet (Anthropic, 2026)	47.81	22.80	-25.01

Table 1: Effects of Time Machine (TM) on the same 61-question set with tool use enabled. All results are averaged over three independent runs. For the w/o TM setting, the web tools have unrestricted access to the open web. All scores are scaled by 100 for readability.

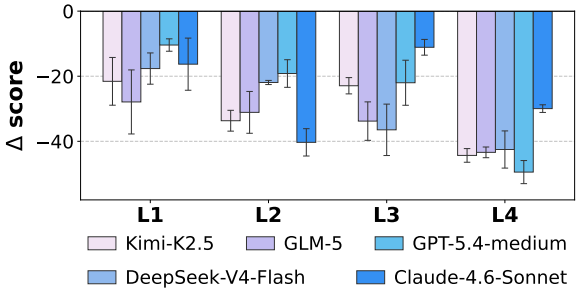


Figure 4: Per-level Δ score (w/ TM − w/o TM, in score points on the 0–100 scale) across five backbones, with ±1 standard deviation error bars over 3 runs per cell. Bars are grouped by FutureX difficulty level and each color denotes one model.

Time Machine to all subsequent offline tool-using experiments.

Where the drop comes from. By breaking the aggregated drop down by question level in Figure 4, we find that (1) Level 4 is the most heavily affected by information leakage, as scores for all backbones fall to a low range between 12 and 29 with the Time Machine (TM) enabled. This sharp decrease occurs because Level 4 consists of open-ended and quantitative questions rather than multiple-choice items, making performance highly dependent on advanced forecasting and reasoning skills once direct web answers are blocked. We also find that (2) Level 1 questions show a much smaller change, and their scores stay within a tight range of 68 to 72. This minor drop of only 10 to 28 points occurs because these few-option questions inherently offer a high random guessing baseline, allowing models to maintain solid scores by leveraging historical baselines, public polls, and prior news to rule out incorrect choices even when future web data is completely hidden. Finally, we find that (3) beyond these two extremes, intermediate levels reveal

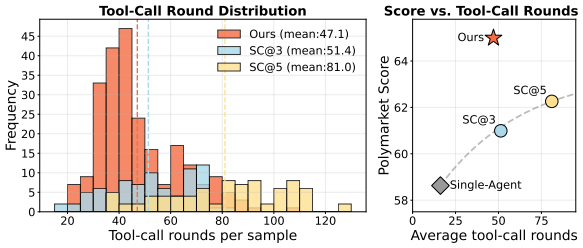


Figure 5: Tool-call efficiency of the multi-agent forecaster compared to self-consistency baselines on Polymarket. **Left**: per-question tool-call distribution across the 80-question pool; dashed lines mark per-method means (Ours 47.1, SC@3 51.4, SC@5 81.0). **Right**: Polymarket score versus average tool-call rounds.

model-specific patterns, where Claude-4.6-Sonnet drops most on Level 2 while its low baseline scores on harder levels limit further decreases, whereas GPT-5.4-medium shows the largest Level 4 drop, indicating its numerical forecasting was heavily driven by open-web leaks rather than robust reasoning. The means and standard deviations are reported in Appendix A.3, and two Level-4 case studies are in Appendix A.4.

Why w/o TM scores are not near 100. Even with unrestricted web access, scores without TM are between 60 and 69 for most models instead of reaching the perfect score of 100. By analyzing the failed cases, we find that three distinct factors limit this performance. Specifically, we observe that (1) some source webpages have been deleted, placed behind paywalls, or removed from search engine indexes over time, meaning this information is no longer retrievable on the live web. We also find that (2) even when the relevant pages still exist, the search queries generated by the agent or the ranking algorithms of the search engine sometimes fail to find them. Finally, we find that (3) even if the agent successfully surfaces a useful page, it occasionally misinterprets the text or chooses not to trust the information. Consequently, both settings have an upper bound on performance, and current agents have substantial room for improvement in both environments. Therefore, the 25 to 37 point drop shown in Table 1 indicates that information leakage significantly inflates open-web performance.

5.3 Offline Results with Agentic Time Machine

Our framework leads on both prediction datasets. On FutureX-Past, our framework reaches a weighted score of 39.51, outperforming the best

Dataset	FutureX-Past					Polymarket
Metric	L1	L2	L3	L4	Score	Score
Models (closed-book, no tools)						
GPT-5.4-medium (OpenAI, 2026)	66.67	21.62	1.01	9.45	15.06	53.76
Claude-4.6-Sonnet (Anthropic, 2026)	62.75	15.65	0.00	0.99	9.78	47.97
Gemini-3.1-Pro (Google, 2026)	54.90	12.83	0.00	3.33	9.36	43.38
Kimi-K2.5 (Kimi Team et al., 2026)	64.70	22.76	1.51	1.48	12.08	42.88
DeepSeek-V4-Flash (DeepSeek AI, 2026)	54.90	14.09	4.69	1.70	10.39	34.79
GLM-5 (GLM-5-Team et al., 2026)	60.79	18.85	1.96	3.33	11.75	41.18
Agents (single-agent w/ tools, Time Machine on)						
GPT-5.4-medium w/ tools	69.98	66.75	<u>12.59</u>	22.52	33.13	58.63
Claude-4.6-Sonnet w/ tools	71.94	43.21	6.91	12.22	22.80	48.21
Gemini-3.1-Pro w/ tools	68.63	62.18	7.90	28.72	33.16	58.45
Kimi-K2.5 w/ tools	68.63	53.72	7.90	17.90	27.13	57.91
DeepSeek-V4-Flash w/ tools	68.63	64.00	12.42	<u>29.07</u>	35.02	48.12
GLM-5 w/ tools	68.17	58.24	8.39	24.90	30.94	59.24
Multi-Agent (Time Machine on)						
GPT-5.4-med-SC@3 (Wang et al., 2023)	82.35	<u>77.89</u>	8.89	23.90	36.04	60.99
GPT-5.4-med-SC@5 (Wang et al., 2023)	**88.24**	**78.82**	8.89	23.90	<u>36.81</u>	<u>62.26</u>
Ours (GPT-5.4-medium)	<u>84.31</u>	71.72	**14.07**	**31.28**	**39.51**	**64.99**

Table 2: Main results on FutureX-Past and Polymarket datasets under Time Machine. For FutureX-Past, we report the four official level scores and their weighted aggregate $0.1\,m_1 + 0.2\,m_2 + 0.3\,m_3 + 0.4\,m_4$. For Polymarket, the score is the unweighted mean set-level F1 over the 80 items. Non-ensemble rows are averaged over three independent runs, while SC@N rows are reported from one ensemble run. The best value in each column is shown in **bold** while the second-best value is <u>underlined</u>.

self-consistency ensemble, SC@5, at 36.81 and the strongest single-agent baseline, DeepSeek-V4-Flash with tools, at 35.02. On Polymarket, our method also obtains the highest score of 64.99, placing it ahead of SC@5 at 62.26 and the top single-agent baseline, GLM-5 with tools, at 59.24. These two datasets differ substantially, as FutureX-Past spans four difficulty levels across diverse forecasting domains, while Polymarket consists entirely of L2-style multi-option questions from real-money prediction markets. The consistent performance gains demonstrate that the planner-solver-aggregator design transfers effectively across different question types instead of overfitting a single benchmark. Furthermore, the detailed performance by level reveals that while self-consistency is competitive on L1 and L2 tasks, our framework establishes its primary lead on the harder L3 and L4 levels, which carry most of the weighted score.

Web tools are the largest single step from closed-book models to forecasting agents. Adding Time Machine-wrapped web tools improves the performance of every backbone on FutureX-Past by 13 to 25 weighted points over its closed-book counterpart, and boosts several backbones by more than 10 points on Polymarket. Even the strongest closed-book model on FutureX-Past, GPT-5.4-medium at 15.06, remains well below the weakest tool-equipped agent, Claude-4.6-Sonnet with tools at 22.80. This performance gap persists even though the Time Machine suppresses post-cutoff and answer-revealing pages, which suggests that the improvement stems from gathering background evidence rather than raw answer lookup. Controlled web access is a fundamental requirement for strong forecasting agents, and the remaining performance gap highlights the need for better evidence collection and organization.

Role decomposition beats majority voting on both score and efficiency. To determine whether our gains come from the multi-agent architecture itself or simply from increased computation, we compare our framework against self-consistency baselines on Polymarket. Here, SC@3 and SC@5 reach scores of 60.99 and 62.26, which are both below our score of 64.99. Figure 5 shows that our

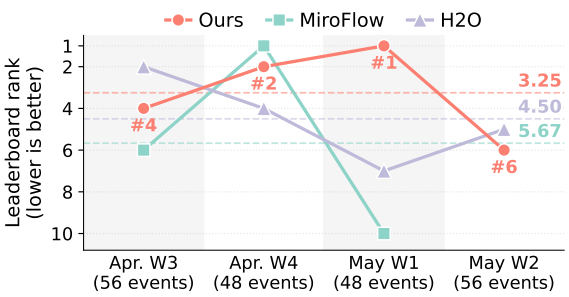


Figure 6: FutureX live leaderboard ranks across four consecutive weekly rounds, where a lower rank is better. Dashed horizontal lines show the average rank of each system over the weeks where it appears.

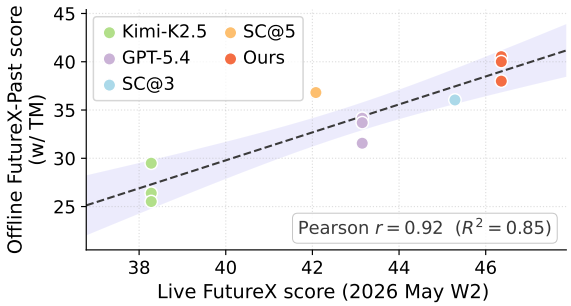


Figure 7: Consistency between offline scores with Time Machine and live FutureX scores from 2026 May W2. Each point corresponds to one run of a method. The dashed line is a linear fit over 11 points.

framework also uses fewer tool-call rounds on average, requiring 47.1 rounds compared to 51.4 for SC@3 and 81.0 for SC@5. While SC@5 gains only 1.27 points over SC@3 at the cost of 58% more rounds, our method improves both accuracy and efficiency. See Appendix C.4 for comparison on FutureX-Past. These comparisons suggest that assigning specific, planner-chosen roles utilizes tool calls much more effectively than repeatedly running the same baseline agent.

5.4 Live Results with FutureX Leaderboard

We submitted our framework to four consecutive FutureX weekly rounds from 2026 Apr. W3 to 2026 May W2. It is listed on the public leaderboard[1] as FutureGPTv1 and referred to as Ours below. We compare against MiroMind's MiroFlow (Su et al., 2026) and the H2O AI Super Agent v1.82[2]. We choose these two systems because they are strong public leaderboard submissions with accompanying public documentation. Full weekly scores, event counts, ranks, and per-level results are reported in Appendix C.3.

Live benchmark results have visible week-to-week variation. FutureX refreshes its question pool every week, so each round has a different set of events, difficulty mix, and resolution status. For this reason, raw scores are not directly comparable across different weeks. Figure 6 therefore presents the public rank results rather than only the absolute score. This view better reflects the live leaderboard setting, where the main comparison is within the same weekly round.

Our system has the best average rank. Across the four submitted rounds, Ours ranks #4, #2, #1, and #6, giving an average rank of 3.25. H2O averages 4.50 over the same four rounds, and MiroFlow averages 5.67 over the first three rounds. Although forecasting events vary by week and performance fluctuates, our system stays in the leading group throughout the period and achieves the strongest average rank among these public baselines.

Offline scores are consistent with live performance. Figure 7 compares each local run's offline FutureX-Past score with TM with the corresponding method's live score in 2026 May W2. The 11 points consist of three runs each for Ours, GPT-5.4-medium, and Kimi-K2.5, plus one run each for SC@3 and SC@5. The relationship is strongly positive, with Pearson $r = 0.92$ and $R^2 = 0.85$. Consequently, the performance achieved with our Agentic Time Machine infrastructure closely mirrors actual forecasting capability under genuine real-world uncertainty, establishing a reliable sandbox for LLM prediction evaluation.

5.5 Framework with Different Base Models

Table 3 investigates whether the planner-solver-aggregator design depends on a single backbone. We run the planner-solver-aggregator framework with three base models and compare it with the corresponding single-agent w/ tools baseline from Table 2. The framework improves all three backbones on both datasets, suggesting that the benefit comes from the role decomposition itself rather than only from GPT-5.4-medium.

6 Conclusion

Future-event forecasting is a strong test for LLM agents, but it is hard to evaluate efficiently. Live

[1]Results are taken from the public leaderboard at https://futurex.live/, accessed on 2026-05-24.

[2]System blog from H2O.ai: https://h2o.ai/blog/2026/h2oai-super-agent-tops-futurex-leaderboard/.

Base model	FutureX-Past			Polymarket		
	Single agent	Framework	Δ	Single agent	Framework	Δ
GPT-5.4-medium	33.13	39.51 ± 1.34	+6.38	58.63	64.99 ± 4.15	+6.36
Claude-4.6-Sonnet	22.80	35.36 ± 2.64	+12.56	48.21	59.03 ± 0.66	+10.82
DeepSeek-V4-Flash	35.02	36.90 ± 2.27	+1.88	48.12	54.48 ± 2.73	+6.36

Table 3: Planner-solver-aggregator framework with different base models. Single-agent scores are the corresponding w/ tools rows in Table 2. Framework scores report mean ± standard deviation over three independent runs. Δ is the framework mean minus the single-agent score. All scores are scaled by 100.

benchmarks are realistic but slow, while historical replay is fast but vulnerable to answer leakage from the modern web. We propose Agentic Time Machine, an LLM-based infrastructure that wraps web tools with a leakage filter and blocks post-cutoff or answer-revealing content before it reaches the agent. This enables faster and less contaminated local replay while keeping open-ended web interaction. We also propose a planner-solver-aggregator framework that splits each question into several analytical angles, gathers evidence in parallel, and reconciles the results into a final forecast. Across FutureX-Past, Polymarket, and the live FutureX leaderboard, our results show that the system is competitive with strong contemporary forecasting agents and improves over strong single agents.

Limitations

Agentic Time Machine provides a controlled approximation of the pre-cutoff environment, but it does not perfectly reconstruct the historical web. Search rankings, snippets, and page availability change over time, so the system cannot guarantee that every item appeared in its exact original form. Additionally, the LLM-based leakage filter introduces a trade-off. It may filter out legitimate pre-cutoff evidence if the text strongly resembles the final answer, while subtle leaks without clear time signals may occasionally pass through. We view Time Machine as a useful but conservative approximation that accelerates historical replay compared to waiting for live resolution, while still preserving a realistic tool interaction pattern.

Our planner-solver-aggregator framework also requires higher inference costs than a single agent due to its multiple solvers and aggregation step. Although efficiency results suggest this extra computation is more effective than simple self-consistency, the framework remains unnecessary for easier questions that require minimal direct evidence.

Ethical Considerations

Our system produces forecasts over future events, including questions about financial markets, macroeconomic indicators, and other high-stakes domains. We note that such predictions can plausibly be used for market speculation or as input to real-world decision-making. The outputs of our framework remain probabilistic and carry substantial uncertainty, and they should not be treated as investment, financial, or policy advice. We use the system as a research artifact for studying LLM agent forecasting and evaluation methodology, and we encourage users to interpret its predictions with appropriate caution and human oversight in any downstream application.

References

Rohan Alur, Bradly C. Stadie, Daniel Kang, Ryan Chen, Matt McManus, Michael Rickert, Tyler Lee, Michael Federici, Richard Zhu, Dennis Fogerty, Hayley Williamson, Nina Lozinski, Aaron Linsky, and Jasjeet S. Sekhon. 2025. Aia forecaster: Technical report. *Preprint*, arXiv:2511.07678.

Anthropic. 2026. Introducing claude sonnet 4.6. https://www.anthropic.com/news/claude-sonnet-4-6. Accessed: 2026-05-12.

Nikhil Chandak, Shashwat Goel, Ameya Prabhu, Moritz Hardt, and Jonas Geiping. 2026. Scaling open-ended reasoning to predict the future. *Preprint*, arXiv:2512.25070.

Hui Dai, Ryan Teehan, and Mengye Ren. 2025. Are llms prescient? a continuous evaluation using daily news as the oracle. *Preprint*, arXiv:2411.08324.

DeepSeek AI. 2026. Deepseek v4 preview release. https://api-docs.deepseek.com/news/news260424. Accessed: 2026-05-12.

GLM-5-Team, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, and 167 others. 2026. Glm-5:

from vibe coding to agentic engineering. *Preprint*, arXiv:2602.15763.

Shashwat Goel, Nikhil Chandak, Arvindh Arun, Ameya Prabhu, Steffen Staab, Moritz Hardt, Maksym Andriushchenko, and Jonas Geiping. 2026. Futuresim: Replaying world events to evaluate adaptive agents. *Preprint*, arXiv:2605.15188.

Google. 2026. Gemini 3.1 pro: A smarter model for your most complex tasks. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/. Accessed: 2026-05-12.

Danny Halawi, Fred Zhang, Chen Yueh-Han, and Jacob Steinhardt. 2024. Approaching human-level forecasting with language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 50426–50468. Curran Associates, Inc.

Chengyuan Jin, Tong Zhou, Yubo Chen, Kang Liu, and Jun Zhao. 2026. Maeps: Multi-agent event prediction system based on human expert team collaboration simulation. *Tsinghua Science and Technology*.

Ezra Karger, Houtan Bastani, Chen Yueh-Han, Zachary Jacobs, Danny Halawi, Fred Zhang, and Philip E. Tetlock. 2025. Forecastbench: A dynamic benchmark of ai forecasting capabilities. In *International Conference on Learning Representations*, volume 2025, pages 93943–93980.

Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, and 307 others. 2026. Kimi k2.5: Visual agentic intelligence. *Preprint*, arXiv:2602.02276.

Ali El Lahib, Ying-Jieh Xia, Zehan Li, Yuxuan Wang, and Xinyu Pi. 2026. Temporal leakage in search-engine date-filtered web retrieval: A retrospective forecasting case study. *Preprint*, arXiv:2602.00758.

Maksym Nechepurenko and Pavel Shuvalov. 2026. Foresight arena: An on-chain benchmark for evaluating ai forecasting agents. *Preprint*, arXiv:2605.00420.

OpenAI. 2026. Introducing gpt-5.4. https://openai.com/index/introducing-gpt-5-4/. Accessed: 2026-05-12.

Shiqian Su, Sen Xing, Xuan Dong, Muyan Zhong, Bin Wang, Xizhou Zhu, Yuntao Chen, Wenhai Wang, Yue Deng, Pengxiang Zhu, Ziyuan Liu, Tiantong Li, Jiaheng Yu, Zhe Chen, Lidong Bing, and Jifeng Dai. 2026. Miroflow: Towards high-performance and robust open-source agent framework for general deep research tasks. *Preprint*, arXiv:2602.22808.

Mingtian Tan, Mike A. Merrill, Zack Gottesman, Tim Althoff, David Evans, and Tom Hartvigsen. 2025. Inferring events from time series using language models. *Preprint*, arXiv:2503.14190.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. *Preprint*, arXiv:2203.11171.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. *Preprint*, arXiv:2504.12516.

Jack Wildman, Nikos I. Bosse, Daniel Hnyk, Peter Mühlbacher, Finn Hambly, Jon Evans, Dan Schwarz, and Lawrence Phillips. 2025. Bench to the future: A pastcasting benchmark for forecasting agents. *Preprint*, arXiv:2506.21558.

Kaiqi Yang, Hang Li, Hongzhi Wen, Tai-Quan Peng, Jiliang Tang, and Hui Liu. 2024. Are large language models (LLMs) good social predictors? In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 2718–2730, Miami, Florida, USA. Association for Computational Linguistics.

Qingchuan Yang, Simon Mahns, Sida Li, Anri Gu, Jibang Wu, and Haifeng Xu. 2025. Llm-as-a-prophet: Understanding predictive intelligence with prophet arena. *Preprint*, arXiv:2510.17638.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. *Preprint*, arXiv:2210.03629.

Chenchen Ye, Ziniu Hu, Yihe Deng, Zijie Huang, Mingyu Derek Ma, Yanqiao Zhu, and Wei Wang. 2024. Mirai: Evaluating llm agents for event forecasting. *Preprint*, arXiv:2407.01231.

Zhiyuan Zeng, Jiashuo Liu, Siyuan Chen, Tianci He, Yali Liao, Yixiao Tian, Jinpeng Wang, Zaiyuan Wang, Yang Yang, Lingyue Yin, Mingren Yin, Zhenwei Zhu, Tianle Cai, Zehui Chen, Jiecao Chen, Yantao Du, Xiang Gao, Jiacheng Guo, Liang Hu, and 12 others. 2025. Futurex: An advanced live benchmark for llm agents in future prediction. *Preprint*, arXiv:2508.11987.

A Time Machine: Mechanics and Empirical Behavior

This section extends Section 3 with the implementation and more empirical analysis of Time Machine on the 61-question controlled subset. Section A.2 describes how the same filtering approach is applied to our two web tools and presents the core rules for each prompt. The quantitative results in §A.3 and §A.5, together with the case studies in §A.4, are computed on the same paired runs (with and without Time Machine, 5 backbones $\times$ 3 runs) that underlie Table 1.

A.1 Web Tools Utilized in our System

Our forecasting agents utilize two retrieval tools. The `search` tool executes a Google query and returns a list of result snippets. Each snippet includes a title, a URL, a short summary text, and a publish date if available. The `visit` tool fetches the body text of a specified URL, which can also summarize the content based on a goal query provided by the agent. Combined, these two tools cover both paths where leaked information might reach the agent. These paths are the brief snippets in the search results and the full page content fetched during a follow-up call.

Both tools are integrated into the agent system using standard JSON function calling schemas.

search tool schema

```
{
  "name": "search",
  "description": "Batched web search.
    Supply an array 'query'; the tool
    retrieves the top 10 results for
    each query in one call.",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "array",
        "items": {"type": "string"},
        "description": "Array of query
          strings. Include multiple
          complementary search queries
          in a single call."
      }
    },
    "required": ["query"]
  }
}
```

visit tool schema

```
{
  "name": "visit",
  "description": "Visit webpage(s) and
    return the summary of the content.",
  "parameters": {
    "type": "object",
    "properties": {
      "url": {
        "type": ["string", "array"],
        "items": {"type": "string"},
        "minItems": 1,
        "description": "The URL(s) of the
          webpage(s) to visit. Can be a
          single URL or an array of URLs."
      },
      "goal": {
        "type": "string",
        "description": "The goal of the
          visit, used to focus the
          per-URL summary."
      }
    },
    "required": ["url", "goal"]
  }
}
```

A.2 Filter Mechanism and Prompts

Time Machine is a tool-output filtering idea, not a fixed pipeline, and serves as the evaluation infrastructure. The leakage filter is agnostic to the specific tool design and can wrap the output of any retrieval or browsing tool before the agent receives the response. In our implementation, the agent has two web tools, `search` and `visit`, so we apply the same LLM-based leakage filter to both tool outputs. The filter follows the two principles:

- **Direct answer leak**: drop any input whose content directly states or reveals the answer to the prediction question, regardless of date.
- **Post-date info leak**: drop any input that carries a time signal (publish date, URL date, in-text event date) later than the event's cutoff time, regardless of content.

The filter applies both principles wherever a tool returns web content. For the `search` tool, it checks the returned titles and snippets. For the `visit` tool, it inspects the fetched page body. This comprehensive filtering guards against two common vulnerabilities. First, an innocuous search title might hide an answer-bearing webpage body. Second, a missing snippet date could be later revealed by an in-page timestamp. Because a leak that is invisible in a brief snippet may still appear in the full page content if the agent visits that URL, checking both tool outputs substantially reduces leakage.

Table 4 contrasts the two tool-output filters along the dimensions where they differ: the LLM and what the agent sees when the filter blocks. We then list the core rules of each prompt.

We use different filter LLMs for the two tool outputs as a speed and cost trade-off. The agent calls `search` much more often than `visit`, and each search call may contain up to ten snippets. We therefore use Kimi-K2.5 for search outputs, since it is faster and cheaper while still strong enough for short snippets. The `visit` tool is called less, but each call contains a much longer page body and can require more careful reading. We therefore use advanced GPT-5.2 for visit outputs. This model choice is an implementation decision; the Time Machine idea only requires that each tool output be checked against the same two leakage principles.

	Search output	Visit output
Filter LLM	Kimi-K2.5	GPT-5.2
Input per call	query, time range, problem, ground truth, $\leq$ 10 snippets	problem, ground truth, cutoff, fetched page body
Effect of block	snippet omitted from the tool's returned list	tool returns `[ACCESS RESTRICTED]`

Table 4: How Time Machine is applied to the two web-tool outputs in our system. The search-output filter processes short snippets in batches; the visit-output filter processes a full page body (up to roughly 12K characters) and is therefore more expensive per call.

Search-side filter. The search-side prompt supplies the judge with the Google `tbs` time-range string, the current real-world date, the prediction problem, the ground truth, and the numbered list of result snippets (title, link, snippet text, and a published date when one is available). Its key rules are:

Search-side filter rules

```
Drop a result if ANY rule applies:

1) Time: The result has a clear published
   date, URL date, title date, or snippet
   date outside the allowed cd_min-cd_max
   window.
2) Spoiler: The title or snippet directly
   resolves the prediction task. This
   applies even when the result has no
   reliable date, and even when the answer
   appears only in the snippet rather than
   the full page.
3) Hindsight: The snippet uses
   result/reporting language that makes a
   still-protected future outcome appear
   settled, known, completed, reported,
   won, lost, announced, confirmed, or
   published.

Exception: keep background, previews,
speculation, or historical context inside
the allowed time window that does not
resolve the task.
```

Visit-side filter. The visit-side prompt supplies the judge with the prediction problem, the ground truth, the cutoff time, the page source URL, and the page body (truncated to roughly 12K characters). Its key rules are:

Visit-side filter rules

```
Rules:
1. Return "LEAKED" if the content directly
   states, strongly implies, or confirms
   the answer to the prediction question.
2. Return "OUT_OF_TIME" only if the content
   itself contains a clear time signal
   (publish/update/event/result time) that
   is later than the cutoff time_point.
3. Return "SAFE" if the content is only
   background, historical context, previews,
   speculation, or the time signal is
   missing/uncertain.
```

A.3 Per-level effect of Time Machine

The aggregate Time Machine drop in Table 1 falls mostly on the retrieval-heavy levels. Every backbone loses 30 to 50 points on Level 4 (quantitative) but only 10 to 28 points on Level 1 (few options). Figure 4 (Section 5.2) plots these per-level Δ values; Table 5 reports the three-run means and standard deviations underlying the figure.

A.4 Case studies

We present two representative cases from the paired GPT-5.4-medium runs that underlie Tables 1 and 5.

Case A: AAPL daily high (L4 quantitative, qid `6851736beb11c800614780df`). Question: "2026-01-23, what will the high of Apple stock (AAPL) be for the day (in US$)?" **GT:** 249.41. The w/o TM agent reproduces $\boxed{249.41}$ exactly under all three trials; the w/ TM agent instead answers 250.90, 250.92, and 252.40. Table 6 traces the second trail of w/ TM and w/o TM. Both filter stages are visible: the search-side filter silently shrinks the result list, and the visit-side filter blocks two of three data sources the agent tries to read.

Case B: Nikkei 225 daily close (L4 quantitative, qid `6851741059f71f006037a1ee`). Question: "2026-01-23, what will be the Nikkei Stock Average (Nikkei 225)'s close for the day?" **GT:** 53,846.87 (agent cutoff: 2026-01-22). The w/o TM agent reproduces $\boxed{53{,}846.87}$ exactly under all three runs; the w/ TM agent instead answers 54,110.50, 53,796.27, and 54,172.09. Table 7 traces the run-1 transcript. Without Time Machine, the first search returns the Yahoo Finance historical-prices page as the top result. Its snippet contains the Jan 23 OHLC row with the close (53,846.87) in plain sight, and the agent answers from the snippet without calling `visit`. With Time Machine, the answer-bearing sources are all blocked. A third-party LinkedIn post quoting a different day's Nikkei close (54,110.5, undated in the snippet) nevertheless slips through, and the agent anchors on this misattributed value.

Model	L1		L2		L3		L4		Weighted	
	w/o	w/	w/o	w/	w/o	w/	w/o	w/	w/o	w/
Kimi-K2.5	$90.2_{\pm 5.6}$	$68.6_{\pm 2.8}$	$87.4_{\pm 2.3}$	$53.7_{\pm 4.0}$	$30.8_{\pm 1.3}$	$7.9_{\pm 1.4}$	$62.2_{\pm 1.5}$	$17.9_{\pm 2.9}$	$60.6_{\pm 0.3}$	$27.1_{\pm 1.7}$
GLM-5	$96.1_{\pm 2.8}$	$68.2_{\pm 7.2}$	$89.4_{\pm 2.9}$	$58.2_{\pm 6.0}$	$42.2_{\pm 2.1}$	$8.4_{\pm 4.3}$	$68.3_{\pm 2.3}$	$24.9_{\pm 2.0}$	$67.5_{\pm 1.4}$	$31.0_{\pm 2.3}$
DeepSeek-V4-Flash	$86.3_{\pm 2.8}$	$68.6_{\pm 7.3}$	$85.9_{\pm 0.5}$	$64.0_{\pm 1.1}$	$48.9_{\pm 3.4}$	$12.4_{\pm 5.0}$	$71.6_{\pm 1.4}$	$29.1_{\pm 4.4}$	$69.1_{\pm 0.9}$	$35.0_{\pm 1.4}$
GPT-5.4-medium	$80.4_{\pm 2.8}$	$70.0_{\pm 0.9}$	$85.9_{\pm 0.2}$	$66.8_{\pm 4.4}$	$34.6_{\pm 3.4}$	$12.6_{\pm 5.2}$	$72.0_{\pm 4.5}$	$22.5_{\pm 1.2}$	$64.4_{\pm 2.6}$	$33.1_{\pm 1.1}$
Claude-4.6-Sonnet	$88.2_{\pm 4.8}$	$71.9_{\pm 3.3}$	$83.5_{\pm 3.1}$	$43.2_{\pm 6.0}$	$18.0_{\pm 1.4}$	$6.9_{\pm 2.8}$	$42.2_{\pm 0.7}$	$12.2_{\pm 1.0}$	$47.8_{\pm 0.6}$	$22.8_{\pm 1.0}$

Table 5: Per-level score with Time Machine off (w/o) and on (w/), reported as mean ± standard deviation across 3 runs. The Δ bars and error bars in Figure 4 are computed from these values.

#	Step
	w/o Time Machine
1	`search`: "AAPL historical Jan 23 2026 high" (+ 2 parallel reformulations)
2	↪ 10 results; the Yahoo Finance result includes the snippet "Jan 23, 2026, 247.32, 249.41, 244.68, 248.04" (OHLC row, high in plain sight)
3	**Answer:** 249.41 *(exact match)*
	w/ Time Machine
1	`search`: "AAPL Jan 2026 news / price / analyst" (+ 2 parallel reformulations)
2	↪ 10 results, all pre-cutoff news / analyst pieces; no Jan 23 OHLC anywhere in the snippets
3	`search`: "AAPL Jan 22 2026 close high low" (+ 2 parallel reformulations)
4	↪ 6 results returned (the search-side filter silently dropped 4 of 10; no Jan 23 OHLC in the surviving snippets)
5	`visit` (×3 in parallel): `investor.apple.com/stock-price`, `stockanalysis.com/stocks/aapl/history/`, `nasdaq.com/. . . /aapl/historical`
6	↪ `investor.apple.com`: `[ACCESS RESTRICTED]`
7	↪ `stockanalysis.com`: `[ACCESS RESTRICTED]`
8	↪ `nasdaq.com`: page passes the filter; its content has Jan 22 OHLC (high 251.00) but no Jan 23 row
9	**Answer:** 250.92 *(wrong; extrapolated from Jan 22 high 251.00, off by 1.51 from GT)*

Table 6: Step-by-step AAPL daily high transcript (Level 4, GPT-5.4-medium run 2); step numbers restart per regime. Runs 1 and 3 get the final answer of 250.90 and 252.40, respectively.

#	Step
	w/o Time Machine
1	`search`: "Nikkei 225 historical Jan 23 2026 close" (+ 2 parallel reformulations)
2	↪ 10 results; the Yahoo Finance result includes the snippet "Jan 23, 2026, 53,898.45, 54,050.84, 53,603.68, 53,846.87" (OHLC row, close in plain sight)
3	**Answer:** 53,846.87 *(exact match)*
	w/ Time Machine
1	`search`: "Nikkei 225 close Jan 22 2026 Reuters" (+ 4 parallel reformulations)
2	↪ 8 results, all pre-cutoff macro articles; no Jan 23 Nikkei close anywhere in the snippets
3	`search`: 4 more Jan 23 Nikkei Reuters queries
4	↪ 9 results returned, among them a LinkedIn post snippet quoting "The Nikkei 225 declined 0.42% to 54,110.5" with no per-day date
5	`visit`: `reuters.com/world/china/global-. . .`, `cnbc.com/2026/01/23/asia-pacific-. . .`
6	↪ `reuters.com`: `[ACCESS RESTRICTED]`
7	↪ `cnbc.com`: `[ACCESS RESTRICTED]`
8	`search`: additional targeted queries using `site:reuters.com`
9	↪ 4 results, all unrelated old articles; no Jan 23 Nikkei data
10	`search`: exact-match `"54,110.5"` Nikkei 225 Jan 23 2026 confirmation
11	↪ 5 results returned, re-quoting the same undated LinkedIn snippet
12	**Answer:** 54,110.50 *(wrong; anchored on undated LinkedIn snippet, off by 263.63 from GT)*

Table 7: Step-by-step Nikkei 225 daily-close transcript (Level 4, GPT-5.4-medium run 1); step numbers restart per regime. Runs 2 and 3 get the final answer of 53,796.27 and 54,172.09, respectively.

A.5 Filter validation and the over-block bias

LLM-based leakage filters face a trade-off between over-blocking and under-blocking. Over-blocking removes legitimate evidence and depresses scores for the wrong reasons; under-blocking lets post-cutoff content reach the agent. To measure where our filter sits on this trade-off, we sampled 150 search-output calls and 150 visit-output calls from one full run (61 questions × 4 levels, GPT-5.4-medium, TM on). Each sample was labeled as Y (should block: snippet or page directly resolves the question, or carries post-cutoff content) or N (should keep: pre-cutoff background, preview, forecast, or unrelated context). Labels were assigned blind to the filter's decision. Table 8 reports counts and metrics for the two tool outputs, with Y as the positive class: TP = blocked among Y; FN = kept among Y (a missed leak); FP = blocked among

N (an over-block); TN = kept among N. P, R, F1 denote precision, recall, and their harmonic mean.

Tool	TP	FN	FP	TN	P	R	F1
Search	24	2	4	120	0.857	0.923	0.889
Visit	41	0	14	95	0.745	**1.000**	0.854

Table 8: Filter validation on 150 labeled samples per tool output. The filter is asymmetric. At the page level, it never misses a true leak in our sample (recall 1.000), at the cost of a precision of 0.745, meaning roughly one in four blocked pages was actually safe. At the snippet level, it misses 2 of 26 leaks (recall 0.923).

The filter is biased toward over-blocking by design: block-when-in-doubt. The remainder of this subsection makes the trade-off concrete via one case from each side, then explains why we prefer this direction.

A miss (false negative). The question "2026-03-19, which movie will be the latest #1 in Box Office Mojo's Domestic Box Office Weekly?" has ground truth *Hoppers*. For one of the agent's queries, Google returned a result with `published_date: Apr 17, 2019` (a stale or spoofed CMS field). Its snippet read: "`Box Office – March 13–15, 2026: HOPPERS, REMINDERS OF HIM, UNDERTONE, & More March 16, 2026.`" The filter prompt covers this case in two rules: Time ("a date in the page path later than `cd_max` ⇒ drop") and Spoiler ("the snippet directly resolves the question ⇒ drop"). The LLM judge missed both. It failed to identify the out-of-range date embedded in the page path. It also missed the spoiler, because reading "March 13–15 #1 = HOPPERS" as resolving the March 19 weekly chart requires knowing that BOM's weekly chart reflects the prior weekend. *Consequence:* a single line of search output names the answer. The agent has no incentive to issue further queries, fetch any page, or reason at all. The prediction task collapses to copying the first capitalized title in the snippet.

An over-block (false positive). The question "Will Bugonia get an Oscar nomination for Best Picture?" has ground truth Yes (`cd_max` = 2026-01-22; the Academy announced nominations on January 23). Two days before the announcement, Variety published *Final Academy Awards Nomination Predictions 2026*. The article listed each category's projected nominees, with Bugonia in the projected Best Picture lineup. The visit-side filter returned `LEAKED` with the reason "Lists Bugonia in Best Picture final nomination predictions, strongly implying it will be nominated." This is a hard call. The Variety prediction does approach answering the question. But it is also exactly the kind of pre-cutoff editorial signal a forecasting agent should weigh. The filter chose the conservative interpretation. *Consequence:* the agent loses one prediction source. It then issues additional queries that surface parallel signals (Awards Watch and Hollywood Reporter's own forecasts, already-announced Golden Globe and BAFTA longlist outcomes, Critics Choice nominations) and reassembles equivalent context for Bugonia. The total signal the agent can weigh converges to roughly the same direction; only the search-and-visit budget is higher.

Why over-blocking is the safer direction. The two failure modes have asymmetrical effects on the agent. A false negative *terminates* reasoning: the leaked snippet (as in the Hoppers example) is itself the answer, and the agent rationally stops searching once it sees one. A false positive only *lengthens* reasoning. The dropped source is usually one of many redundant copies of the same pre-cutoff context that the web carries about most questions, and the agent recovers the missing signal by issuing more queries or visiting parallel pages. In the limit, a false positive costs search budget; a false negative costs the entire scientific value of the w/ TM run, because the agent never had to predict. We therefore tune the filter toward the first failure mode. The precision cost in Table 8 is the price of treating "block-when-in-doubt" as a safe default for a contamination filter.

B Local Datasets

This appendix introduces the two resolved local datasets used in our offline experiments. The FutureX-Past subset is used for the Time Machine validation and controlled local comparison in the main paper. The Polymarket subset adds a separate high-difficulty multiple-choice forecasting set. Both subsets are formatted as pre-resolution prediction tasks; ground truth is used only for offline scoring.

Licenses and terms of artifacts. FutureX-Past is released by the FutureX authors on the Hugging Face Hub (https://huggingface.co/datasets/futurex-ai/Futurex-Past) under the Apache-2.0 license; we cite the original dataset and paper and use only a 61-question subset for

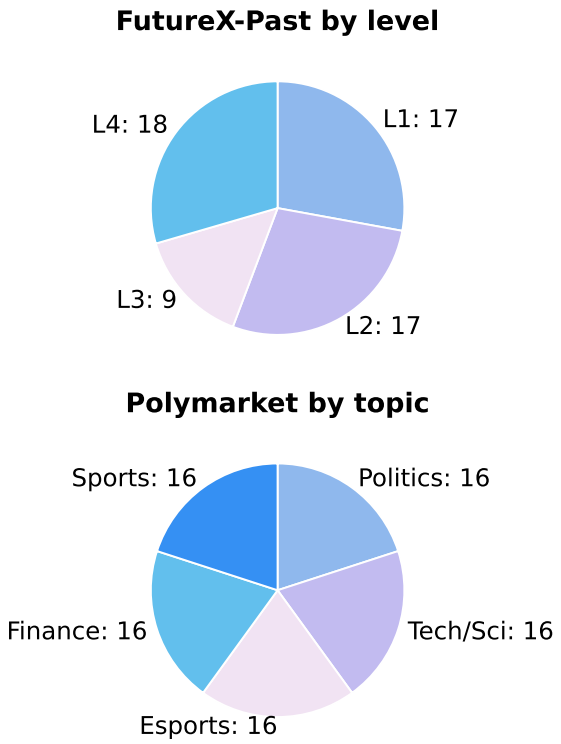


Figure 8: Observed composition of the two local subsets. FutureX-Past covers all four benchmark levels, while the Polymarket subset contains exactly 16 items from each of five topics.

research evaluation. Our Polymarket subset is derived from resolved public market/event metadata collected through Polymarket's public Gamma API (https://docs.polymarket.com/api-reference/introduction), which provides unauthenticated access to market and event data. We use this data only for research evaluation, document the collection procedure and access date.

Named individuals and content. All named persons in both subsets are public figures appearing as the subject of public prediction questions (candidates, athletes, executives, etc.), and no PII (contact, financial, or protected-attribute information) is collected. We preserve original Polymarket question wording verbatim to remain aligned with the canonical ground truth; the authors manually inspected all 80 Polymarket and 61 FutureX-Past items and found no hate speech, harassment, or sexually explicit content. Anonymization is not applicable because forecasting questions are inherently about identifiable public events and actors.

Subset	Construction summary
FutureX-Past	61 resolved FutureX historical questions, sampled from https://huggingface.co/datasets/futurex-ai/Futurex-Past, observed level counts are 17/17/9/18 for L1-L4.
Polymarket	80 resolved Polymarket questions sampled from a 4,573-item balanced parent pool built from the public Gamma API; the subset is evenly split across five topics.

Table 9: High-level summary of the two local evaluation subsets.

B.1 FutureX-Past

FutureX-Past is the historical archive of the FutureX benchmark. Its entries are resolved prediction tasks with a question prompt, ground truth, resolution time, and difficulty level. Because these questions are already resolved, they are not used as evidence of live forecasting ability by themselves.

The FutureX-Past subset contains 61 questions. The observed level distribution is 17/17/9/18 for Levels 1–4 respectively (Figure 8). Level 1 mostly consists of binary or small categorical questions, Level 2 contains larger multiple-choice questions, Level 3 contains open-ended ranking or compositional questions, and Level 4 contains quantitative or open-ended questions scored by the official graded metric. The subset spans resolution dates from 2026-01-15 to 2026-04-06.

Representative FutureX-Past examples

L1. Islanders vs. Oilers. *GT:* `Islanders`.
L2. Q1 S&P 500 Performance. *GT:* `A`.
L3. 2026-03-19, value in Chinese Yuan of 100 units of British Pound at the central parity rate. *GT:* `915.04`.
L4. 2026-01-23, high of Apple stock (AAPL) for the day, in US$. *GT:* `249.41`.

Local corrections. We compared the 61 local rows against the corresponding rows in https://huggingface.co/datasets/futurex-ai/Futurex-Past. Most differences are formatting-only changes in prompt whitespace. Three items were manually corrected in the local subset. Two are corrected ground-truth labels for multiple-choice questions; the third fixes a mismatch between the required answer format in the prompt and the stored ground-truth label.

Manual corrections in the FutureX-Past subset

`6957ba8a03568a006853e828`. For "How will the winner of the Portugal presidential election win?", the ground truth is corrected from `A` (only candidate above 50% in round one) to `B` (wins first round and the runoff).

`69b7f816d596fb005d43a317`. For "F1: Who will win the 2026 Aramco Japanese Grand Prix?", the ground truth is corrected from `A` (George Russell) to `B` (Kimi Antonelli).

`69624695e87498005daa01d6`. The prompt asks for `\boxed{Succeeded}` or `\boxed{Backfired}`, but the stored ground truth was `A`; we correct it to `Succeeded` to match the prompt's literal answer format.

B.2 Polymarket

The Polymarket dataset is constructed from resolved markets and events collected through Polymarket's public Gamma API. We used the keyset-paginated event endpoint because Polymarket events often group several related binary submarkets, which can be naturally converted into multi-option, sometimes multi-answer, prediction questions. The parent collection is restricted to resolved items between 2026-01-01 and 2026-05-08.

For each event, we convert the market or grouped event into a labeled multiple-choice question. Single-market items use the market outcomes as options. Grouped events with multiple Yes/No submarkets use each submarket as one option, preferring concise option text from `groupItemTitle`, then `shortOutcomes`, then the market question or slug. Ground truth labels are inferred from final outcome prices: a label is marked correct when the corresponding winning outcome price is at least 0.99. For grouped events, all submarkets whose Yes price crosses this threshold contribute labels to the ground-truth set.

The parent dataset applies filtering and balancing before the 80-item evaluation subset is drawn. The construction keeps only resolved items with at least one confident ground-truth label, filters noisy or high-frequency tags such as recurring and very short crypto markets from the main pool, favors longer-horizon multi-option grouped events, and caps short-horizon and binary supplements so they do not dominate the benchmark. The resulting balanced parent pool has 4,573 items. The 80-item subset is a focused hard set: it has exactly 16 items in each of five topics, no binary items, no short-horizon items, 67 multi-answer items, option counts from 8 to 26, and a mean heuristic difficulty score of 49.7. Table 10 summarizes the construction-time heuristic used to rank and balance candidate items.

Signal	Score rule	Effect
Option count	$+18$ for ≥ 12; $+14$ for 8–11; $+10$ for 4–7; $+3$ for $= 3$; -10 for ≤ 2	Rewards larger answer spaces.
Event shape	$+8$ for grouped events; $+6$ for multiple correct labels	Rewards questions that combine several submarkets.
Horizon	$+12$ for ≥ 1 day; $+4$ for 6–24 hours; -14 for < 6 hours	Downweights very short markets.
Topic/tags	$+8$ for politics, finance/economy, tech/science, or world news; penalties for recurring, up-or-down, and crypto-price tags	Favors less mechanical domains and filters noisy high-frequency markets.

Table 10: Polymarket difficulty-score heuristic. The score is used only during dataset construction for ranking and balancing candidate items.

Representative Polymarket examples

Politics. Question: What will Trump say this week? (April 26) **Options:** A. Make America Great Again; B. Transgender; C. Trump Strait / Strait of Trump; D. DoorDash / McDonald's; E. Shit; F. Genius; G. Excursion / Journey; H. The War Is Over; I. Kamala; J. UFC / Dana White; K. Pope / Leo; L. Tough Cookie; M. Death to America; N. My Father / Fred; O. Boy oh boy; P. Crypto / Bitcoin. **GT:** `A, B, F, G, H, I, P`.

Tech/science. Question: Where will it snow this weekend? (January 24–26) **Options:** A. New York City; B. Dallas; C. Miami; D. Seattle; E. Washington D.C.; F. Phoenix; G. Philadelphia; H. Atlanta; I. Boston; J. Pittsburgh. **GT:** `A, B, E, G, I, J`.

Esports/games. Question: BLAST Open Rotterdam 2026: Which teams make it to the Grand Final? **Options:** A. Vitality; B. Furia; C. Mouz; D. Team Falcons; E. Parivision; F. Team Spirit; G. The MongolZ; H. Natus Vincere; I. FaZe; J. Aurora; K. Team Liquid; L. B8; M. NRG; N. Ninjas in Pyjamas; O. Tyloo; P. 9z. **GT:** `A, H`.

Finance/economy. Question: What will BlackRock say during their next earnings call? **Options:** A. Performance Fee; B. Aladdin; C. Digital Asset; D. ETF Inflow; E. Private market; F. Crypto / Bitcoin; G. Record AUM; H. Stablecoin. **GT:** `A, B, C, E`.

Sports. Question: Angers SCO vs. Paris Saint-Germain FC - Player Props **Options:** A. Goalscorer: Ousmane Dembele; B. Goalscorer: Goncalo Ramos; C. Goalscorer: Bradley Barcola; D. Goalscorer: Desire Doue; E. Goalscorer: Khvicha Kvaratskhelia; F. Goalscorer: Lee Kang-in; G. Goalscorer: Senny Mayulu; H. Goalscorer: Ibrahim Mbaye; I. Goalscorer: Fabian Ruiz; J. Goalscorer: Nuno Mendes; K. Goalscorer: Vitinha; L. Goalscorer: Joao Neves; M. Goalscorer: Achraf Hakimi; N. Goalscorer: Amine Sbai; O. Goalscorer: Lanroy Machine; P. Goalscorer: Peter Prosper. **GT:** `F, G`.

C Additional Experiment Details

C.1 Models Configuration

In our paper, we utilize six large language models: GPT-5.4-medium, Claude-4.6-Sonnet, Gemini-3.1-

Pro, Kimi-K2.5, DeepSeek-V4-Flash and GLM-5. The first three of them are closed-source models with no public model size information. For the remaining models, Kimi-K2.5 has 1T total parameters with 32B activated parameters, DeepSeek-V4-Flash has 284B total parameters with 13B activated parameters, and GLM-5 has 744B total parameters with 40B activated parameters.

C.2 Software Dependencies

Our agents and evaluation scripts rely on a small set of standard open-source Python packages: `openai` for OpenAI-compatible chat completion calls, `tenacity` for request retry, `requests` and `aiohttp` for web tool HTTP calls, `tiktoken` for token-budget truncation, `tqdm` for progress bars, `tabulate` for score summary printing, and `qwen-agent` for its base tool registration utility. We do not use any other NLP-specific evaluation libraries (e.g., NLTK, SpaCy, ROUGE) because the FutureX scoring code is implemented directly from the public repo. All packages are used at their default settings; we do not configure any non-default model, threshold, or hyperparameter inside them.

C.3 Week-by-week FutureX live leaderboard results

Table 11 lists, for each of the four consecutive FutureX weekly rounds we submitted to, the per-level breakdown and final weighted score for our system (FutureGPTv1) alongside the two strongest contemporaneous external submissions (MiroMind's MiroFlow and the H2O AI Super Agent v1.82). Our system ranks between #1 and #6 across the four weeks. The live leaderboard scores are not directly comparable across weeks because each round uses a fresh question pool and has a different mix of resolved levels. In particular, the L3/L4 score of May W1 is 0.00 for any listed system at the leaderboard snapshot, so every submission's weighted score is driven only by the L1 and L2 terms, $0.1\,m_1 + 0.2\,m_2 \leq 30$, making the absolute scores in that week much lower than in the other rounds.

C.4 Complementary Efficiency Results

Figure 9 reports the per-question tool-call distribution on the left and a score-versus-rounds plot on the right on FutureX-Past dataset. Two features stand out: 1) our framework averages 34.1 rounds per question, sitting between SC@3 (25.1) and SC@5 (42.3). Its distribution is also tighter than SC@5, which has a long right tail extending past 90 rounds. 2) The three baselines line up on a concave efficiency curve with clearly diminishing returns. SC@5 only gains 0.8 weighted points over SC@3 for about 68% more rounds. Our framework lies clearly above this curve. It beats SC@5 on both axes, with a higher score and fewer rounds. Together, this shows that splitting a question into planner-chosen angles is a more efficient way to spend tool calls than running the same agent many times and voting.

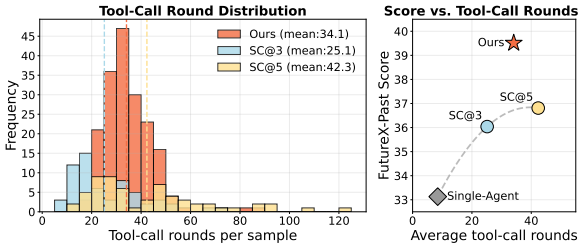


Figure 9: Tool-call efficiency of the multi-agent forecaster compared to self-consistency baselines. **Left**: per-question tool-call distribution across the 61-question pool; dashed lines mark per-method means (Ours 34.1, SC@3 25.1, SC@5 42.3). **Right**: weighted score versus average tool-call rounds.

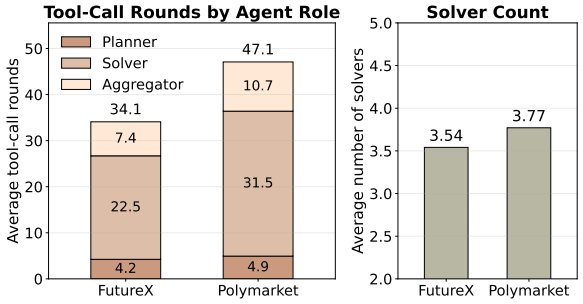


Figure 10: Tool-call cost of our framework on FutureX and Polymarket. **Left**: average tool-call rounds per question, decomposed into planner, solvers, and aggregator. **Right**: average number of solvers selected by the planner per question.

Here, we also break down our framework's tool budget by agent role and compare the two datasets. Figure 10 reports the average tool-call rounds spent by the planner, solvers, and aggregator on each question, together with the average number of solvers the planner chooses. On FutureX, a question takes 34.1 rounds on average, while on Polymarket it takes 47.1. The planner cost is almost the same on both datasets (4.2 vs 4.9 rounds), so the extra budget on Polymarket comes from the solvers (22.5 to 31.5) and the aggregator (7.4 to 10.7). The planner also tends to spawn slightly more solvers

Week (events)	Model	System	Rank	L1	L2	L3	L4	Score
Apr. W3 (56)	Claude-Sonnet-4.6	H2O AI Super Agent v1.82	2	89.47	50.45	50.21	52.80	55.22
	GPT-5.4	**Ours** (FutureGPTv1)	**4**	84.21	50.30	42.64	50.98	51.67
	GPT-5	MiroFlow	6	68.42	50.02	46.52	52.07	51.63
Apr. W4 (48)	GPT-5	MiroFlow	1	42.86	72.78	57.77	36.04	50.59
	GPT-5.4	**Ours** (FutureGPTv1)	**2**	42.86	69.17	54.07	32.70	47.42
	Claude-Sonnet-4.7	H2O AI Super Agent v1.82	4	42.86	66.06	48.88	34.74	46.06
May W1 (48)	GPT-5.4	**Ours** (FutureGPTv1)	**1**	91.67	86.18	0.00	0.00	26.40
	Claude-Sonnet-4.6	H2O AI Super Agent v1.82	7	87.50	72.60	0.00	0.00	23.27
	GPT-5	MiroFlow	10	79.17	75.95	0.00	0.00	23.11
May W2 (56)	Claude-Sonnet-4.6	H2O AI Super Agent v1.82	5	100.00	43.57	41.78	38.31	46.57
	GPT-5.4	**Ours** (FutureGPTv1)	**6**	87.50	57.14	37.78	37.12	46.36
	GPT-5.4	GPT-5.4-SC@3 (GPT@3)	7	87.50	57.14	37.77	34.45	45.29
	GPT-5.4	GPT-5.4 w/ search (GPT-search)	8	75.00	50.00	39.55	34.45	43.15
	GPT-5.4	GPT-5.4-SC@5 (GPT@5)	9	75.00	50.00	36.00	34.45	42.08
	Kimi-K2.5	Kimi-K2.5 (Simple ReAct)	17	87.50	42.14	29.24	30.82	38.28

Table 11: Weekly FutureX live leaderboard scores for our submission (FutureGPTv1) and the two strongest systems, accessed at 2026-05-24. For May W2, we also include four additional baselines: GPT-5.4 with search, GPT-5.4-SC@3, GPT-5.4-SC@5, and Kimi-K2.5 with tools.

on Polymarket (3.77 vs 3.54).

Polymarket questions are harder for two reasons. First, they are often multi-part markets: a single question can bundle several player props or game outcomes, so each solver has to gather more facts and the aggregator has to cross-check more sub-claims against box scores, lineups, and odds. Second, the relevant information is scattered across many sources such as team news, sportsbook lines, recap articles, and statistics sites, which forces both solvers and the aggregator to issue more search and visit calls. FutureX questions, in contrast, more often center on a single authoritative source such as an official ranking or a published index, which is easier to resolve.

C.5 Case Studies for Planner-Solver-Aggregator Framework

We analyze two trajectories through our multi-agent framework on FutureX-Past. In both cases, the final answer is not directly available from the agent's tool calls. The correct prediction therefore requires reasoning over multiple sources, rather than simple retrieval. Table 12 and Table 13 summarize the outputs of the planner, solvers, and aggregator for each run.

Case A: Q1 2026 S&P 500 quarter return (L2, qid `69a2e39e5692ef005cdbf2d8`). The question presents nine non-overlapping return ranges for the S&P 500 price return on March 31, 2026, where choice A represents a return below 0%. Because the Time Machine blocks the final closing value, the agent must deduce the correct range from earlier market data. Table 12 outlines the actions of each role. All three solvers independently select choice A using different analytical paths, and the aggregator confirms the underlying data before final alignment.

Case B: NASA GISTEMP December 2025 anomaly (L2, qid `6941510e41b9d1005effa734`). The question is about the December 2025 global land-ocean temperature anomaly relative to the 1951 to 1980 baseline, where choice A contains values below 1.095°C. The official NASA GISTEMP data files are completely unreadable through the visit tool, forcing the agents to infer the answer from alternative climate datasets instead. Table 13 details the actions of each role. Unlike the uniform consensus in the previous case, the three solvers disagree completely. Two solvers rely on historical intuition that the period was exceptionally warm, whereas the aggregator performs cross-dataset calculations to override these incorrect estimates. This specific conflict indicates why vanilla voting fails on complex forecasting tasks: a simple majority vote over the three conflicting answers results in a tie and selects the correct choice only by chance. In contrast, our framework reaches the correct answer because the aggregator independently verifies the cross-source evidence.

C.6 Additional Coding-Agent Baselines

To further assess whether our gains transfer beyond forecasting-specific agent designs, we addi-

Role	Key action / finding	Pick	Conf.
Planner	Decomposes the task into three angles, namely a near-resolution market-state tracker, a historical Q1 distribution analyst, and a late-quarter macro / event-risk analyst.	–	–
Solver 1 (market path)	Anchors Dec. 31 2025 close (6,845.50) and Mar. 30 2026 close (6,343.72); computes quarter-to-date return of -7.33%; notes that escaping bucket A on the final session would require a $+7.91\%$ one-day rally.	A	0.94
Solver 2 (history)	Independent prior shortlist {A, E, F} on quarter-distribution grounds; verifies the same Dec. 31 and Mar. 30 anchors via Yahoo Finance snippets; concludes that the -7.33% gap is far outside ordinary quarter-end churn.	A	0.92
Solver 3 (macro)	Iran/Middle-East oil shock pushed oil sharply higher, Fed easing expectations were repriced lower because of inflation risk, and tariff/trade uncertainty remained a headwind; all unfavorable for a late rebound.	A	0.95
Aggregator	Cross-confirms the two anchor closes from independent search snippets (CNBC, Morningstar/Dow Jones, Yahoo for Mar. 30; Yahoo and syndicated close reports for Dec. 31); reproduces the same $+7.91\%$ rally threshold and finds no event capable of producing one.	**A**	0.94

Table 12: Multi-agent trajectory for the Q1 2026 S&P 500 question (qid `69a2e39e5692ef005cdbf2d8`, with ground truth A). All three solvers independently reach A via complementary analytical paths; the aggregator re-verifies the anchors and concurs.

tionally compare our framework with three general coding-agent systems: Codex[3], OpenHands[4], and DeepAgents[5]. All systems use the same underlying GPT-5.4 model and the same two Time Machine-wrapped web tools under the same evaluation protocol. These supplementary comparisons are conducted as single-run experiments, and the corresponding point estimates are reported in Table 14.

Under this experimental setup, the general coding agents are less suited to event forecasting than our forecasting-specific framework. Codex and OpenHands are designed around software-development workflows, where an agent inspects a workspace, runs commands, updates files, and verifies a concrete result. DeepAgents adds general planning, subagents, filesystem context, and code execution for complex tool use. These capabilities are useful for software-oriented tasks, but they do not directly match the demands of forecasting. Forecasting requires broad search, source comparison, and synthesis of uncertain signals. In our observed trajectories, coding agents sometimes pursue a direct answer or repeat similar searches after filtered results or inaccessible pages. By contrast, our framework assigns diverse evidence paths, investigates them in parallel, and aggregates the resulting views. This alignment with the forecasting task helps explain its stronger results than both self-consistency and coding-agent baselines under the same Time Machine setting.

[3] https://openai.com/codex/
[4] https://github.com/OpenHands/openhands
[5] https://github.com/langchain-ai/deepagents

Role	Key action / finding	Pick	Conf.
Planner	Decomposes the task into three angles, namely an official NASA release reader, a cross-dataset triangulator using NOAA and Copernicus, and a revision-risk watchdog.	–	–
Solver 1 (NASA reader)	NASA tables and CSVs not retrievable through visit; reads the indirect 'fifth-warmest December' framing as a moderately-warm signal and places mass on bin D without baseline conversion.	D	0.47
Solver 2 (cross-dataset)	NOAA December 2025 anomaly $1.05^{\circ}\mathrm{C}$ above 20th-century mean and Copernicus December anomaly $0.49^{\circ}\mathrm{C}$ above 1991–2020 (both ranked fifth-warmest December); mapping to NASA's 1951–1980 baseline gives ≈ 1.0–$1.1^{\circ}\mathrm{C}$, just below the A/B boundary.	A	0.57
Solver 3 (revision)	Initial prior anchored on broad 'late-2025 was hot' regime intuition, giving an E/F/G shortlist before evidence; NASA revision-cadence snippets address only update timing, not the central value.	F	0.43
Aggregator	Verifies NOAA and Copernicus exact-month values; notes the annual NASA–NOAA gap is only about $+0.02^{\circ}\mathrm{C}$, not enough to lift the December value above the $1.095^{\circ}\mathrm{C}$ threshold; downweights Solver 1 (only an ordinal rank, no numeric anomaly to compare against this threshold) and Solver 3 (not month-specific).	**A**	0.58

Table 13: Multi-agent trajectory for the NASA GISTEMP December 2025 question (qid `6941510e41b9d1005effa734`, with ground truth A). The three solvers split 1–1–1 across D, A, F; the aggregator's own cross-dataset arithmetic overrides the two warm-prior angles and selects A.

Dataset	FutureX-Past					Polymarket
Method	**L1**	**L2**	**L3**	**L4**	**Score**	**Score**
GPT-5.4 w/ tools	69.98	66.75	<u>12.59</u>	22.52	33.13	58.63
GPT-5.4-SC@3	82.35	<u>77.89</u>	8.89	<u>23.90</u>	36.04	60.99
GPT-5.4-SC@5	**88.24**	**78.82**	8.89	<u>23.90</u>	<u>36.81</u>	<u>62.26</u>
Codex	76.47	44.17	5.93	12.59	23.30	58.03
OpenHands	76.47	43.53	2.96	14.72	23.13	50.96
DeepAgents	76.47	55.49	5.93	7.04	23.34	56.05
Ours	<u>84.31</u>	71.72	**14.07**	**31.28**	**39.51**	**64.99**

Table 14: Comparison with general coding-agent baselines on FutureX-Past and Polymarket under the same Time Machine-based tool setting.